

Tugas 1 Data Mining

author : Naufal Aziz Ferdian 124450116

1. Clening data

```
import pandas as pd
from openpyxl import load_workbook
from openpyxl.styles import Alignment, Font, PatternFill
from openpyxl.utils import get_column_letter

FILE_CSV = "../data/train.csv"
FILE_SEBELUM = "../output/dataset_sebelum_cleaning.xlsx"
FILE_SESUDAH = "../output/dataset_setelah_cleaning.xlsx"

KOLOM_ID = "PassengerId"
KOLOM_NOMINAL = ["Survived", "Pclass", "Sex", "Embarked"]
KOLOM_NUMERIK = ["Age", "Fare"]
KOLOM_DIPAKAI = [KOLOM_ID] + KOLOM_NOMINAL + KOLOM_NUMERIK

def log(pesan):
    print(f" - {pesan}")

# 1. Baca Data
def baca_data_csv(path):
    df = pd.read_csv(path)
    print(f"Data dibaca : {df.shape[0]} baris dan {df.shape[1]} kolom")
    return df

def pilih_kolom(df):
    df = df.copy()
    dibuang = [k for k in df.columns if k not in KOLOM_DIPAKAI]
    log(f"kolom tidak dipakai: {dibuang}")
```

```

    return df[KOLOM_DIPAKAI]

# 2. small function untuk cleaning
def hapus_duplicated(df):
    df = df.copy()
    sebelum = len(df)
    df = df.drop_duplicates(subset=[KOLOM_ID]).drop_duplicates()
    log(f"Baris duplikat dihapus : {sebelum - len(df)}")
    return df

def standarkan_teks(df):
    df = df.copy()
    df["Sex"] = df["Sex"].str.strip().str.lower()
    df["Embarked"] = df["Embarked"].str.strip().str.lower()
    log("Teks kolom Sex dan Embarked dirapikan")
    return df

def ubah_tipe_data(df):
    df = df.copy()
    for kolom in KOLOM_NUMERIK:
        df[kolom] = pd.to_numeric(df[kolom], errors="coerce")
    for kolom in KOLOM_NOMINAL:
        df[kolom] = df[kolom].astype("object")
    log("Tipe data diseuaiaka")
    return df

def isi_nilai_kosong_numerik(df):
    df = df.copy()
    # Mengisi nilai kosong dengan median
    kosong_age = df["Age"].isna().sum()
    median_kelompok = df.groupby(["Pclass", "Sex"])["Age"].transform("median")
    df["Age"] = df["Age"].fillna(median_kelompok).fillna(df["Age"].median())

    # fare dengan median keseluruhan
    kosong_fare = df["Fare"].isna().sum()
    df["Fare"] = df["Fare"].fillna(df["Fare"].median())

    return df

def isi_nilai_kosong_nominal(df):

```

```

df = df.copy()
for kolom in KOLOM_NOMINAL:
    kosong = df[kolom].isna().sum()
    if kosong:
        modus = df[kolom].mode()[0]
        df[kolom] = df[kolom].fillna(modus)
        log(f"{kolom}: {kosong} nilai kosong diisi modus ('{modus}'))")
return df

def tangani_outlier(df, kolom="Fare"):
    """Batasi nilai ekstrem dengan aturan IQR (winsorizing)."""
    df = df.copy()
    q1, q3 = df[kolom].quantile([0.25, 0.75])
    iqr = q3 - q1
    batas_bawah, batas_atas = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    jumlah = ((df[kolom] < batas_bawah) | (df[kolom] > batas_atas)).sum()
    df[kolom] = df[kolom].clip(lower=batas_bawah, upper=batas_atas)
    log(
        f"{kolom}: {jumlah} outlier dibatasi ke "
        f"[{batas_bawah:.2f}, {batas_atas:.2f}] (aturan IQR)"
    )
    return df

def bulatkan_numerik(df):
    df = df.copy()
    df["Age"] = df["Age"].round(1)
    df["Fare"] = df["Fare"].round(2)
    return df

def bersihkan_data(df):
    return (
        df.pipe(pilih_kolom)
        .pipe(hapus_duplicated)
        .pipe(standarkan_teks)
        .pipe(ubah_tipe_data)
        .pipe(isi_nilai_kosong_nominal)
        .pipe(isi_nilai_kosong_numerik)
        .pipe(tangani_outlier, kolom="Fare")
        .pipe(bulatkan_numerik)
        .reset_index(drop=True)
    )

```

```

)

def ringkasan_kualitas(df, judul):
    print(f"\n[{judul}]\n")
    print(f"  Jumlah baris : {len(df)}")
    print(f"  Jumlah kolom : {df.shape[1]}")
    print(f"  Duplikat      : {df.duplicated().sum()}")
    kosong = df.isna().sum()
    print("  Nilai kosong per kolom:")
    for nama, jumlah in kosong.items():
        print(f"    {nama:<12} {jumlah}")

def validasi(df):
    assert len(df) >= 100, "Baris kurang dari 100"
    assert df.shape[1] >= 5, "Kolom kurang dari 5"
    assert df.isna().sum().sum() == 0, "Masih ada nilai kosong"
    assert df.duplicated().sum() == 0, "Masih ada duplikat"
    assert (df["Age"] > 0).all(), "Ada Age yang tidak valid"
    assert (df["Fare"] >= 0).all(), "Ada Fare yang tidak valid"
    print("\nValidasi lolos: tidak ada nilai kosong atau duplikat.")
    return df

# Tahap 4: menyimpan ke Excel dengan tampilan rapi
def simpan_excel(df, path, nama_sheet="Data"):
    df.to_excel(path, index=False, sheet_name=nama_sheet)

    wb = load_workbook(path)
    ws = wb[nama_sheet]
    font_biasa = Font(name="Arial", size=10)
    font_header = Font(name="Arial", size=10, bold=True, color="FFFFFF")
    isi_header = PatternFill("solid", start_color="305496")

    for baris in ws.iter_rows():
        for sel in baris:
            sel.font = font_biasa
    for sel in ws[1]:
        sel.font = font_header
        sel.fill = isi_header
        sel.alignment = Alignment(horizontal="center")

```

```

for i, kolom in enumerate(df.columns, start=1):
    lebar = max(len(str(kolom)), df[kolom].astype(str).str.len().max()) + 3
    ws.column_dimensions[get_column_letter(i)].width = min(lebar, 40)

ws.freeze_panes = "A2"
wb.save(path)
print(f"File disimpan: {path}")
return df

# Program utama
def main():
    mentah = baca_data_csv(FILE_CSV)
    ringkasan_kualitas(mentah, "SEBELUM cleaning")
    simpan_excel(mentah, FILE_SEBELUM, nama_sheet="Data mentah")

    print("\nProses cleaning:")
    bersih = bersihkan_data(mentah)

    ringkasan_kualitas(bersih, "SESUDAH cleaning")
    validasi(bersih)
    simpan_excel(bersih, FILE_SESUDAH, nama_sheet="Data bersih")

if __name__ == "__main__":
    main()

```

Data diabaca : 891 baris dan 12 kolom

```

[SEBELUM cleaning]
Jumlah baris : 891
Jumlah kolom : 12
Duplikat      : 0
Nilai kosong per kolom:
PassengerId   0
Survived      0
Pclass        0
Name          0
Sex           0
Age          177
SibSp         0
Parch         0

```

Ticket	0
Fare	0
Cabin	687
Embarked	2

File disimpan: ../output/dataset_sebelum_cleaning.xlsx

Proses cleaning:

- kolom tidak dipakai: ['Name', 'SibSp', 'Parch', 'Ticket', 'Cabin']
- Baris duplikat dihapus : 0
- Teks kolom Sex dan Embarked dirapikan
- Tipe data disesuaikan
- Embarked: 2 nilai kosong diisi modus ('S')
- Fare: 116 outlier dibatasi ke [-26.72, 65.63] (aturan IQR)

[SESUDAH cleaning]

Jumlah baris : 891

Jumlah kolom : 7

Duplikat : 0

Nilai kosong per kolom:

PassengerId	0
Survived	0
Pclass	0
Sex	0
Embarked	0
Age	0
Fare	0

Validasi lolos: tidak ada nilai kosong atau duplikat.

File disimpan: ../output/dataset_setelah_cleaning.xlsx

Ini adalah untuk cleaning data

hasil korelasi

```
import numpy as np
import pandas as pd
from itertools import combinations
from scipy.stats import chi2_contingency

df = pd.read_excel("../output/dataset_setelah_cleaning.xlsx")
```

```

nominal = ["Survived", "Pclass", "Sex", "Embarked"]
numerik = ["Age", "Fare"]

def cramers_v(a, b):
    tabel = pd.crosstab(df[a], df[b])
    chi2, p, dof, _ = chi2_contingency(tabel, correction=False)
    v = np.sqrt(chi2 / (tabel.values.sum() * (min(tabel.shape) - 1)))
    return chi2, p, v

# Korelasi nominal: Chi-Square dan Cramer's V
print("KORELASI NOMINAL")
for a, b in combinations(nominal, 2):
    chi2, p, v = cramers_v(a, b)
    print(f"{a} - {b}: chi2={chi2:.3f}, p={p:.4g}, Cramer's V={v:.3f}")

# Korelasi numerik: Pearson
print("\nKORELASI NUMERIK (Pearson)")
print(df[numerik].corr(method="pearson").round(4))

```

KORELASI NOMINAL

```

Survived - Pclass: chi2=102.889, p=4.549e-23, Cramer's V=0.340
Survived - Sex: chi2=263.051, p=3.712e-59, Cramer's V=0.543
Survived - Embarked: chi2=25.964, p=2.301e-06, Cramer's V=0.171
Pclass - Sex: chi2=16.971, p=0.0002064, Cramer's V=0.138
Pclass - Embarked: chi2=122.641, p=1.457e-25, Cramer's V=0.262
Sex - Embarked: chi2=12.917, p=0.001567, Cramer's V=0.120

```

KORELASI NUMERIK (Pearson)

	Age	Fare
Age	1.0000	0.1794
Fare	0.1794	1.0000

Normalisasi dengan z-score

```

import pandas as pd

df = pd.read_excel("../output/datsert_setelah_cleaning.xlsx")

```

```

numerik = ["Age", "Fare"]

# Z-Score: (x - rata-rata) / simpangan baku
for kolom in numerik:
    df[kolom + "_zscore"] = ((df[kolom] - df[kolom].mean()) / df[kolom].std()).round(4)

print(df.head(10))
print("\nPemeriksaan (mean harus 0, std harus 1):")
print(df[["Age_zscore", "Fare_zscore"]].agg(["mean", "std"]).round(4))

df.to_excel("hasil_normalisasi.xlsx", index=False)

```

	PassengerId	Survived	Pclass	Sex	Embarked	Age	Fare	Age_zscore \
0	1	0	3	male	s	22.0	7.25	-0.5346
1	2	1	1	female	c	38.0	65.63	0.6680
2	3	1	3	female	s	26.0	7.92	-0.2339
3	4	1	1	female	s	35.0	53.10	0.4425
4	5	0	3	male	s	35.0	8.05	0.4425
5	6	0	3	male	q	25.0	8.46	-0.3091
6	7	0	1	male	s	54.0	51.86	1.8706
7	8	0	3	male	s	2.0	21.08	-2.0379
8	9	1	3	female	s	27.0	11.13	-0.1588
9	10	1	2	female	c	14.0	30.07	-1.1359

	Fare_zscore
0	-0.8201
1	2.0304
2	-0.7874
3	1.4186
4	-0.7811
5	-0.7610
6	1.3581
7	-0.1448
8	-0.6307
9	0.2941

Pemeriksaan (mean harus 0, std harus 1):

	Age_zscore	Fare_zscore
mean	-0.0	-0.0
std	1.0	1.0